

Efficient Balanced Networks on FPGA

Mostafa Ibrahim

Electrical and Computer Engineering

University of California, Davis

Davis, United States

mokibrahim@ucdavis.edu

Abstract—Efficient Balanced Networks (EBNs) are biologically inspired neuromorphic computing models designed for energy-efficient and robust online learning. Inspired by the adaptive behavior of simple nervous systems, EBNs offer advantages over traditional spiking neural networks (SNNs), such as maintaining a dynamic balance between excitation and inhibition for improved stability and adaptability. While previous work has implemented EBNs using mixed-signal circuits, these designs face challenges like device mismatch, noise sensitivity, and limited scalability. This work addresses these challenges by demonstrating a fully digital FPGA-based implementation of the EBN model on the Nexys A7 100T FPGA, using a damped harmonic oscillator as a test case. The architecture employs modular components—neuron core, synaptic core, spike filter, desired dynamic module, memory manager, and controller module—that operate in parallel with precise synchronization to achieve real-time online learning and adaptive behavior. By discretizing continuous EBN computations into synchronized, clock-driven steps and leveraging the FPGA’s parallel processing capabilities, this implementation preserves the core principles of EBNs while ensuring scalability, flexibility, and energy efficiency. The results lay the groundwork for the feasibility of FPGA-based EBNs for real-world neuromorphic applications, highlighting both the potential and the challenges that need to be addressed in future research.

I. INTRODUCTION

Efficient Balanced Networks (EBNs) are biologically inspired neuromorphic computing models designed for energy-efficient and robust online learning. These models are inspired by the nervous systems of simple organisms, such as fruit flies, which exhibit sophisticated behaviors like sensorimotor pattern generation and navigation despite having limited neural resources. EBNs implement principles of predictive coding and adaptive control by continuously updating synaptic weights based on real-time error feedback, allowing the system to learn and predict dynamic behaviors, such as those of a damped harmonic oscillator. This approach mimics biological neural networks by focusing on minimizing prediction errors and adapting to changes efficiently, making it suitable for hardware implementations where energy efficiency and real-time processing are critical [1].

Unlike conventional deep neural networks (DNNs), convolutional neural networks (CNNs), and recurrent neural networks (RNNs), which rely on large datasets and extensive offline training, EBNs excel at online learning—adapting weights dynamically based on incoming data. This makes them ideal for real-time applications such as Brain-Machine Interfaces (BMIs), robotic control systems, and other low-

power neuromorphic tasks where adaptability and energy efficiency are essential. Potential applications include neural signal decoding, real-time filtering for prosthetics, and adaptive robotics [3, 4]. Conventional neural networks implemented on GPUs are impractical for these tasks due to their high energy consumption, large form factor, and reliance on batch processing, which makes real-time adaptability difficult. In contrast, hardware-based neuromorphic implementations using FPGAs or mixed-signal circuits offer real-time processing, compact size, and energy efficiency.

Previous work has demonstrated EBN implementations using mixed-signal circuits [2]. These designs benefit from low power consumption and continuous data representation. However, mixed-signal implementations face significant challenges, such as device mismatch, noise sensitivity, and limited scalability due to the complexity of integrating analog and digital components. To overcome these issues, this work explores a fully digital FPGA-based implementation. Digital FPGAs offer flexibility, modularity, and precise control over computations, making them well-suited for scalable and adaptive neuromorphic systems.

This work focuses on demonstrating the feasibility of implementing the EBN model on the Nexys A7 100T FPGA. Unlike mixed-signal approaches, which benefit from continuous data representation, an FPGA-based design requires converting EBN computations into synchronized, clock-driven steps. The goal is to preserve the real-time online learning capabilities and adaptive behavior of EBNs while leveraging the advantages of digital hardware, such as flexibility, modularity, and scalability. By addressing the unique challenges of digital implementation, this work showcases the potential of FPGA-based EBNs for energy-efficient, real-time neuromorphic applications.

II. PROBLEM DEFINITION

A. Implementing EBNs on Digital FPGAs

Adapting Efficient Balanced Networks (EBNs) to a fully digital FPGA architecture presents unique challenges compared to mixed-signal implementations. While mixed-signal designs naturally support continuous data representation and low-power operation, they face limitations such as device mismatch, noise sensitivity, and scalability issues [2]. In contrast, FPGA-based implementations offer flexibility, modularity, and scalability but require discretizing continuous EBN computations into synchronized, clock-driven steps.

The core challenge is to preserve EBN’s adaptive behavior and real-time online learning within these digital constraints. This involves breaking down continuous computations—such as neuron potential updates and synaptic weight adjustments—into finite steps synchronized with the FPGA’s clock cycles. The digital design must leverage the FPGA’s parallel processing capabilities to run operations concurrently, minimizing stalls and ensuring computations remain synchronized across modules (e.g., neuron core, synaptic core, and spike filter).

Maintaining synchronization and efficient scheduling between these modules ensures weight updates are dynamically applied during each cycle, preserving the real-time learning capability that characterizes EBNs. This approach enables FPGA implementations to achieve the accuracy, stability, and adaptability required for energy-efficient, real-time neuromorphic applications, overcoming the limitations of mixed-signal designs.

B. Online Learning Implementation

A defining feature of EBNs is their ability to perform real-time online learning by continuously updating synaptic weights based on incoming data and error feedback. Unlike traditional neural networks that rely on extensive offline training, EBNs dynamically adapt to new inputs during operation. Implementing this capability on an FPGA requires maintaining continuous weight updates without interrupting ongoing computations.

To achieve real-time online learning, the system integrates predictive coding and adaptive control principles within the digital architecture. During each computation cycle, synaptic weights are adjusted based on real-time error feedback. The controller module plays a crucial role in managing synchronization signals and coordinating data flow between the neuron core, synaptic core, and spike filter. This precise coordination ensures that weight updates occur seamlessly, supporting the adaptive learning behavior needed for applications like Brain-Machine Interfaces (BMIs) and adaptive robotic systems.

C. Advantages of EBNs Over Traditional SNNs

While traditional spiking neural networks (SNNs), such as the ODIN neuromorphic processor [5], offer potential for neuromorphic computing, EBNs provide several advantages that make them more suitable for real-time adaptive hardware applications.

1) Balance and Stability:

EBNs are specifically designed to maintain a dynamic balance between excitatory and inhibitory signals. This balance prevents issues such as runaway excitation or network inactivity, ensuring the system remains stable and robust even under dynamic conditions. This stability is crucial for real-time applications where consistent performance is necessary, such as BMIs and robotic control.

2) Efficient Online Learning:

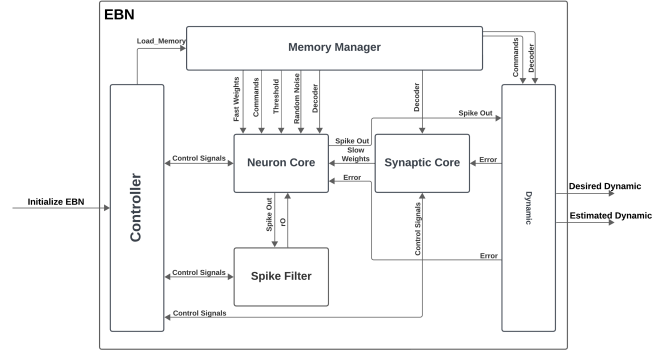


Fig. 1. EBN Block Diagram

Unlike traditional NNs, which often require pre-training and static weight updates, EBNs integrate predictive coding and adaptive control to enable continuous online learning. This real-time adaptability allows EBNs to learn and adjust synaptic weights on the fly, making them ideal for tasks that require immediate responsiveness to new data, such as decoding neural signals or real-time sensorimotor control.

3) Energy Efficiency:

EBNs achieve energy efficiency by focusing on minimizing prediction errors and maintaining a balanced network state. This results in fewer unnecessary spikes and optimized computational effort. For hardware implementations, such as those on FPGAs or custom neuromorphic chips, this efficiency translates to reduced power consumption, making EBNs well-suited for low-power, embedded systems where energy constraints are critical.

III. ARCHITECTURAL DESIGN

The FPGA implementation of the Efficient Balanced Networks (EBN) model employs a modular architecture that distributes computations across four key modules: the neuron core, synaptic core, spike filter, and the desired dynamic module (Fig. 1). This architecture is designed to achieve real-time online learning and maintain a balanced regime of excitation and inhibition through parallel processing, structured memory management, and precise synchronization. The overall system operates in the following steps:

1) Initialization:

Upon receiving the initialization signal, the controller sends a load signal to both the memory manager and the synaptic core. The memory manager loads threshold values (T), decoder values (D), random commands (c), and fast weights (Wf) from BRAMs. Simultaneously, the synaptic core preloads the first 64 slow weights (W) to prepare for the upcoming neuron membrane potential calculations.

2) Compute Neuron Potentials:

Once the data has been loaded from the BRAMs, the neuron core begins calculating membrane potentials

for 64 neurons upon receiving a start signal from the controller. Based on these potentials, the spike-out sub-module generates output spikes (O).

3) Update Filtered Spikes:

The spike filter processes the output spikes to update the filtered spike values (rO), which feed back into the neuron core for subsequent potential calculations.

4) Manage Synaptic Weights:

The synaptic core is responsible for supplying the slow weights (W) to the neuron core. During learning, it updates these weights based on error feedback from the desired dynamic module.

5) Generate Dynamics and Error Feedback:

The desired dynamic module calculates the expected dynamics (x) and estimated dynamics (xest). The difference between these values produces an error signal (e), which guides the weight updates in the synaptic core.

6) Final Evaluation:

After the desired number of learning iterations has been completed, the controller disables learning and runs the system for one final cycle without learning to evaluate the final network state and performance.

By discretizing the continuous computations of EBNs into clock-driven steps, this design preserves adaptive learning behavior while leveraging the FPGA's strengths in parallel processing and structured memory management. This approach overcomes the limitations of mixed-signal designs and enables a scalable, energy-efficient architecture suitable for real-time neuromorphic applications.

A. Neuron Core

The neuron core updates the membrane potentials (V) for 64 neurons and identifies which neuron generates an output spike (O). The spike-out submodule handles the generation of output spikes, while the LIF-AU submodule computes the membrane potential using the equation below:

$$\begin{aligned} V(:, t+1) = & (1 - \lambda_V \cdot \Delta t) \cdot V(:, t) + D' \cdot c(:, t) \\ & + W_f \cdot O(:, t) + W \cdot rO(:, t) \cdot \Delta t \\ & + \text{nzMemb} \cdot \text{randn}(N, 1) \cdot \text{gainD}^2 \\ & + K \cdot D' \cdot e(:, t) \end{aligned} \quad (1)$$

The LIF-AU submodule performs matrix operations to compute the potential of a single neuron over 64 cycles. The calculation involves contributions from slow weights (W), filtered spikes (rO), fast weights (Wf), and the decoder (D), along with the leak and noise terms. An additional 3 cycles are used for synchronization, threshold comparison, and ensuring data stability, resulting in a total of 67 cycles per neuron. For the final neuron calculation (64th neuron), the process extends to 70 cycles.

As each neuron's potential is calculated, it is compared against a predefined threshold. If the potential exceeds the threshold, the neuron is flagged as a potential spiker. Among all flagged neurons, the one with the highest potential is

selected as the spiker, in accordance with the EBN model's rule of allowing only one spike per time step.

B. Spike Filter Module

The spike filter updates the filtered spikes (rO), which depend on the output spikes (O) generated by the neuron core. The update follows the below equation:

$$rO(:, t+1) = (1 - \lambda \cdot \Delta t) \cdot rO(:, t) + O(:, t+1) \quad (2)$$

While the neuron core calculates membrane potentials, the spike filter simultaneously processes the initial part of rO. This computation takes 64 cycles and is synchronized with the neuron core's iterative calculations, ensuring seamless operation and preventing idle time between the modules.

$$(1 - \lambda \cdot \Delta t) \cdot rO(:, t) \quad (3)$$

Once a spike is generated by the neuron core, the spike filter updates only the spiking neuron's position in rO by adding the spike value "1". This avoids the need for a full matrix addition, significantly reducing computational overhead. The spike filter finalizes its calculations as the neuron core completes its last potential. This ensures rO is fully updated and ready for the next computation cycle, with minimal delay.

C. Synaptic Core

The synaptic core manages the slow weights (W), providing them to the neuron core, and updating them during the learning phase based on equation (4). The synaptic core uses a finite state machine (FSM) with three states:

$$W = W + \text{eta}_W \cdot dt \cdot (rO(:, t) \cdot (D' \cdot e(:, t)))' \quad (4)$$

1) Preload State:

At startup or after a reset iteration, the synaptic core preloads 64 weights from BRAM. This ensures that the neuron core has access to the weights needed before the calculations begin.

2) Normal Operation State:

During normal operation (no learning), the synaptic core supplies weights to the neuron core one at a time until it loads the 64 weights associated with that specific neuron. This operation takes place one cycle after the neuron core's start signal. While the neuron core calculates potentials, the synaptic core preloads the next set of weights to avoid delays.

3) Learn State:

During the learning phase, the synaptic core updates the slow weights (W) using the error signal (e) and filtered spikes (rO). To ensure the system operates without stalls, learning begins while the neuron core calculates the potential for the last neuron. Since the weight update depends on both rO and e, and these values are also being updated in parallel for the next cycle, the system employs a double-buffering approach where two versions of rO and e are maintained:

- Current Version: Used in the potential calculation for the current cycle.

- Old Version: Used for the weight update (W).

This separation allows the learning process to run concurrently with the updates to rO and e , ensuring that weight updates do not stall the system. As a result, the system can compute W while simultaneously updating the next cycle's values for rO and e .

D. Desired Dynamic Module

The module computes the expected dynamics (x), estimated dynamics ($xest$), and error (e) for the system, these signals take one cycle to complete the calculation. Unlike modules that involve matrix multiplications, this module relies solely on basic multiplications and additions.

The expected dynamic (x), mimics the behavior of a damped harmonic oscillator, producing expected outputs for velocity $x(1)$ and position $x(2)$. This computation is defined independently of the rest of the system and evolves based on the following equation:

$$x(:, t+1) = x(:, t) + desDyn \quad (5)$$

$$desDyn = (3 \cdot [-dynprm \cdot x(2) - x(1); x(1)] \cdot dt + c) \quad (6)$$

The estimated dynamics are generated by applying the decoder (D) to the system's output spikes (O). The computation is given by:

$$xest(:, t+1) = (1 - \lambda \cdot \Delta t) \cdot xest(:, t) + D \cdot O(:, t+1) \quad (7)$$

Unlike conventional matrix multiplication, this computation takes advantage of the sparsity of O , where only one neuron can spike at any given time, with all other values being zero. Instead of performing a full multiplication, the system checks for a spike using the spike-flg signal and determines the spiking neuron's position using the spike-pos signal. When a spike occurs, the decoder values corresponding to the spiking neuron's position are applied to calculate the estimated dynamics. This is implemented by selectively assigning decoder values based on the spike's position. If no spike occurs, the contribution to $xest$ is zero, avoiding unnecessary calculations. This optimized approach significantly reduces computational overhead by eliminating redundant multiplications for non-spiking neurons, making the computation both efficient and aligned with the modular design philosophy of the system.

The error signal represents the difference between the expected dynamics (x) and the estimated dynamics ($xest$). This error signal is fed back into the system to adjust the slow weights (W) during learning. It ensures the system iteratively improves its estimation of the dynamics, aligning $xest$ more closely with x .

To ensure alignment with the neuron core's calculations, a synchronization signal ($next$) is generated by the matrix multiplication module within the LIF-AU submodule. The $next$ signal is set one cycle before the calculation of a neuron potential (pot) is completed, triggering the dynamic and estimated dynamic modules to start their computations. The results of these computations are stored in flip-flops, ensuring that x and $xest$ are available one cycle after the next signal

is set, synchronizing with the calculation of the next neuron potential.

This design is inherently scalable. If pipeline stages are added to the multipliers in the future to improve speed, the next signal can be adjusted to trigger earlier by the required number of pipeline stages. This ensures synchronization remains intact without introducing stall cycles, enabling faster operation while maintaining efficient data flow.

E. Memory Manager

The memory manager handles data loading and distribution from BRAM, ensuring that data is available to the appropriate modules when needed. Three data sets are fully loaded at startup and remain unchanged throughout the operation: 64 threshold values, 128 decoder values, and 64 random noise values.

The remaining data sets, including random commands and fast weights, are also unchanged but are provided at specific stages of the calculation depending on the system's state. Fast weights are supplied in the same manner as the slow weights, aligned with each neuron's potential calculation. Random commands are provided once per system cycle, after all 64 neuron potentials have been calculated. This ensures the system operates with fresh inputs for the next computation cycle and maintains synchronization across all modules.

F. Controller Module

The controller module coordinates the operation of the EBN system by managing control signals, synchronizing module interactions, and ensuring smooth transitions between states. Its primary role is to sequence tasks across the system while adapting to the current operational state, such as initialization, normal operation, and learning.

At startup, the controller responds to an initialization signal ($init$ -signal) to begin system operations. From there it sends a signal to the memory manager to initialize the data loading from the BRAMs. Once the loading process is flagged as complete ($load$ -complete), the controller signals the neuron core to start processing.

The controller generates synchronization signals ($start$ -neuron, $start$ -spike-filter) to align the neuron core and spike filter modules. It monitors the progress of neuron potential calculations and spike filtering, ensuring that no module stalls or operates prematurely. A waiting signal ($wait$ -spike) is generated when required to ensure that spike output processing aligns with the system's timing.

During the learning phase, the controller enables the synaptic core's weight update process by asserting the learning enable signal ($learn$ -en). To avoid stalling the system, learning begins concurrently with the last neuron potential calculation. The controller ensures synchronization by leveraging signals from the neuron core, such as $neuron$ -processing-done, and manages state transitions for the synaptic core as updates proceed.

The controller tracks learning iterations and signals the end of an iteration by generating an internal reset signal ($reset$ -iteration). This reset re-initializes all modules except for the

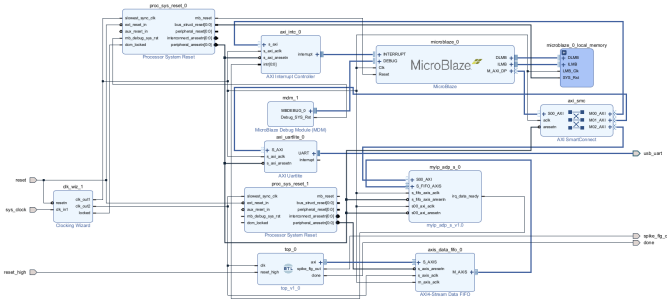


Fig. 2. Vivado Block Design

synaptic memory, which retains the updated weights. After completing the specified number of iterations, the controller transitions the system to a post-learning state, disabling learning and preparing for final data capture.

IV. IMPLEMENTATION TOOLS

The implementation of the EBN model on the Nexys A7 100T FPGA was achieved through an integrated workflow using Vivado and Vitis.

Vivado was the primary tool for the synthesis, implementation, and simulation of the design. The EBN module was encapsulated within a top module responsible for interfacing with the AXI bus for data output. A block design was used to set up the MicroBlaze processor, which operated at 100 MHz (Fig. 2). This required a FIFO to handle clock domain crossing, as the EBN module ran at 12.5 MHz. Floating-point operations were initially designed manually but later replaced with Vivado's floating-point IPs, including the fused multiply-add (FMA) core for matrix multiplication.

Vitis was employed to program the MicroBlaze processor and manage data communication between the FPGA and external systems. The main program implemented an interrupt handler to retrieve and store data efficiently, avoiding polling delays. Captured data, including dynamics (x) and estimated dynamics (x_{est}), was transmitted via UART for post-processing.

V. RESULTS

A. Precision Analysis

After 100 learning iterations, the estimated and expected dynamics for velocity were compared $x_{est}(1)$ $x(1)$ (Fig. 3), as well as the estimated and expected dynamics for position $x_{est}(2)$ and $x(2)$ (Fig. 4).

While the estimated dynamics align with the expected dynamics initially, deviations become more pronounced as learning progresses. The limited precision of bfloat16 is a likely contributor, as small weight updates in the learning equation (4), are not fully captured due to insufficient precision, especially as weight values grow.

With the learning rule module updated to work with FP32, Fig. 5 comparing velocity $x(1)$ to its estimate $x_{est}(1)$ shows a closer match during the initial 2,000 time steps.

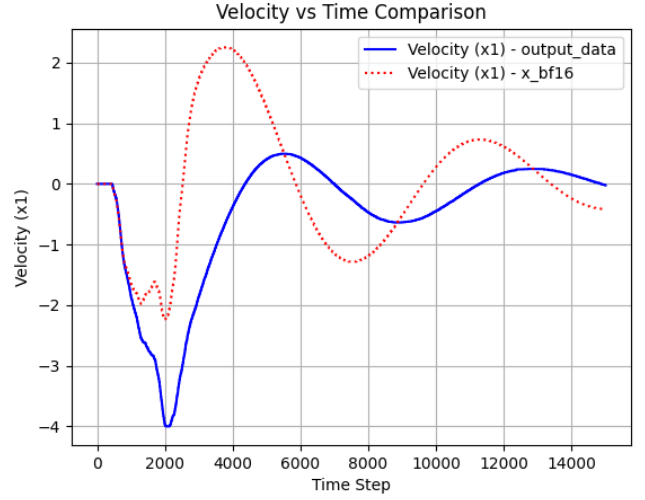


Fig. 3. Bfloat 16 expected Vs. Estimate Velocity.

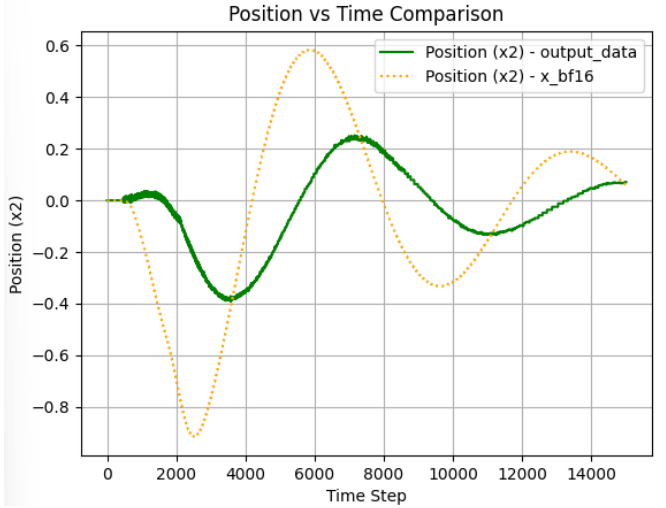


Fig. 4. Bfloat 16 expected Vs. Estimate Position.

FP32 better captures smaller weight updates during the early phases of learning, leading to improved alignment with expected dynamics compared to bfloat16. However, as learning iterations continue, alignment deteriorates, similar to bfloat16, indicating limitations in precision for extended learning.

B. Resource Utilization

The design's resource utilization on the Nexys A7 100T FPGA is summarized in Fig. 6. The design leveraged DSP blocks for floating-point computations and BRAM for storing weights and dynamic data. The design demonstrated efficient resource utilization, leaving room for enhancements such as pipelining to increase performance.

VI. CONCLUSION

This work demonstrated the feasibility of implementing the Efficient Balanced Networks (EBN) model on the Nexys A7

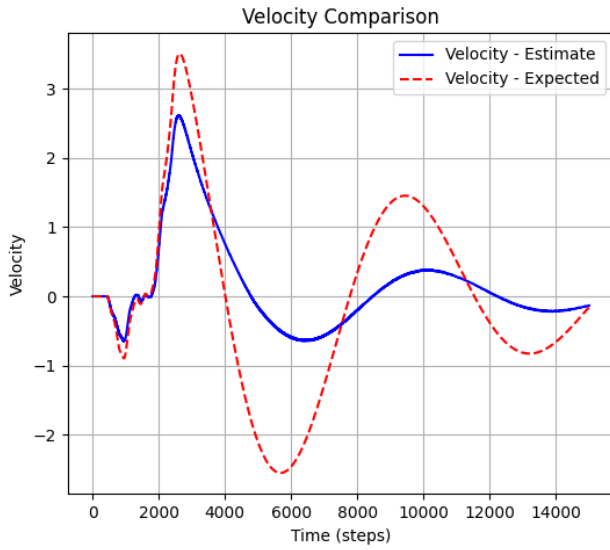


Fig. 5. FP32 expected Vs. Estimate Velocity.

Resource	Estimation	Available	Utilization %
LUT	10634	63400	16.77
FF	11326	126800	8.93
BRAM	34.50	135	25.56
DSP	46	240	19.17
IO	39	210	18.57
BUFG	1	32	3.13

Fig. 6. Nexys A7 100T Utilization.

100T FPGA, using a damped harmonic oscillator as a proof of concept. The modular design incorporated key components such as the neuron core, synaptic core, spike filter, desired dynamic module, and memory manager, with parallel computations ensuring efficient use of FPGA resources. The system was validated through extensive simulations and hardware testing, capturing expected dynamics with bfloat16 precision while maintaining synchronization and scalability.

Key findings revealed that the system's learning convergence was limited over extended iterations. While bfloat16 provided resource efficiency, it may have contributed to difficulties in capturing small weight updates, hindering long-term alignment with expected dynamics. Comparisons with FP32 demonstrated improved early-phase accuracy, but convergence challenges remained. However, it is not yet conclusive whether precision limitations are the primary cause of these issues or if other factors, such as architectural constraints or synchronization, are contributing to the observed behavior. Further investigation, including testing higher precision formats like FP64 or exploring architectural optimizations, is necessary to identify and address the root cause of these limitations.

The resource utilization of the design showcased the efficiency of the architecture, with potential for performance improvements through techniques like pipelining to increase clock speeds. The system's modular structure offers a scalable

framework for future applications, including larger networks or real-time data processing.

Future work will focus on pinpointing the cause of the convergence challenges, addressing precision-related constraints, optimizing architectural performance, and enhancing the real-time adaptability of the EBN model. This study lays a solid foundation for advancing neuromorphic computing hardware, bridging the gap between biologically inspired models and practical, energy-efficient implementations.

REFERENCES

- [1] Alireza Alemi, Christian Machens, Sophie Deneve, and Jean-Jacques Slotine. "Learning nonlinear dynamics in efficient, balanced spiking networks using local plasticity rules". In: Proceedings of the AAAI Conference on Artificial Intelligence. Vol. 32. 1. 2018.
- [2] J. Buchel, J. Kakon, Perez, Michel, and G. Indiveri, "Implementing Efficient Balanced Networks with Mixed-Signal Spike-Based Learning Circuits," IEEE, 2021. Accessed: Dec. 10, 2024. [Online]. Available: <https://www.zora.uzh.ch/id/eprint/217495/1/2010.14353.pdf>
- [3] J. Dethier, P. Nuyujukian, S. I. Ryu, K. V. Shenoy, and K. Boahen, "Design and validation of a real-time spiking-neural-network decoder for brain-machine interfaces," Journal of Neural Engineering, vol. 10, no. 3, p. 036008, Apr. 2013, doi: <https://doi.org/10.1088/1741-2560/10/3/036008>.
- [4] I. Karageorgos et al., "Hardware-Software Co-Design for Brain-Computer Interfaces," 2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA), Valencia, Spain, 2020, pp. 391-404, doi: 10.1109/ISCA45697.2020.00041.
- [5] C. Frenkel, M. Lefebvre, J. -D. Legat and D. Bol, "A 0.086-mm² 12.7-pJ/SOP 64k-Synapse 256-Neuron Online-Learning Digital Spiking Neuromorphic Processor in 28-nm CMOS," in IEEE Transactions on Biomedical Circuits and Systems, vol. 13, no. 1, pp. 145-158, Feb. 2019, doi: 10.1109/TBCAS.2018.2880425.